

رسالة محمد



یادگیری ماشین

مدرس: محمدرضا محمدی

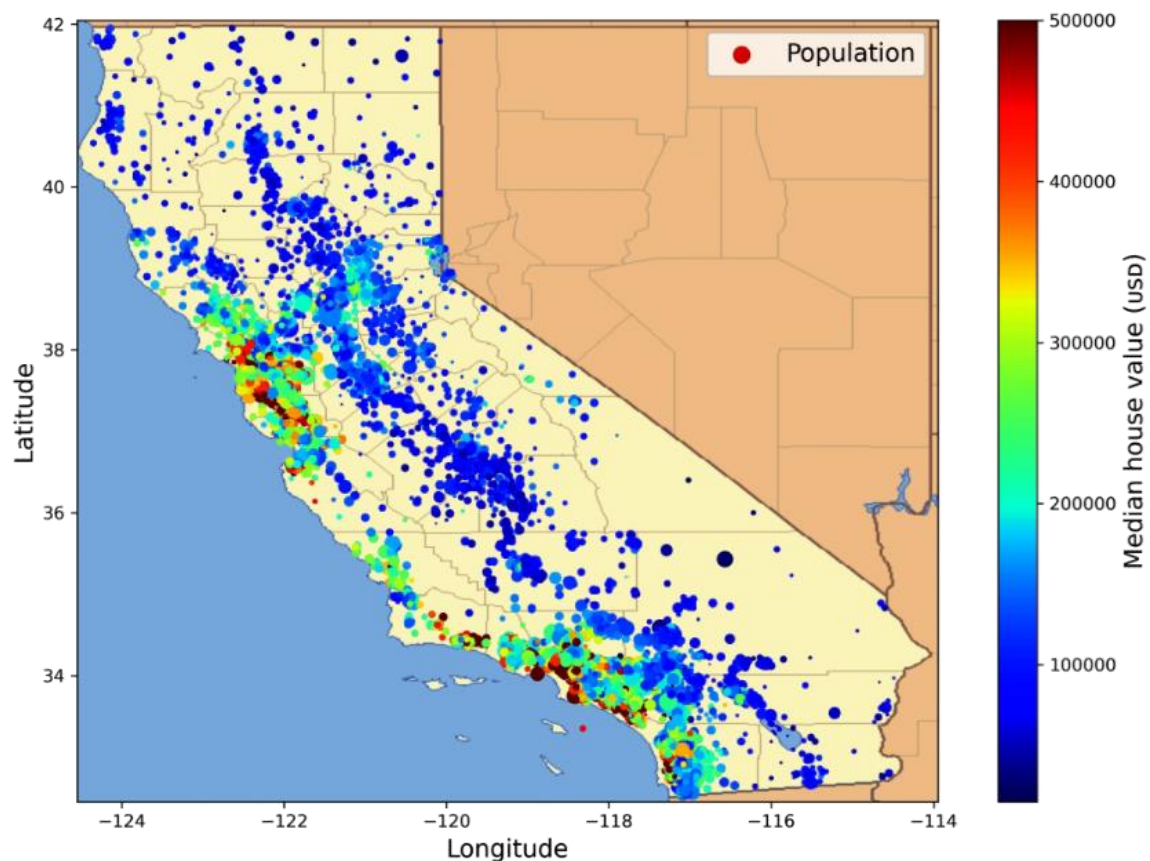
زمستان ۱۴۰۱

پروژه یادگیری ماشین انتها به انتها

End-to-End Machine Learning Project

پروژه یادگیری ماشین انتها به انتها

- می‌خواهیم با یک مثال ساختگی (تخمین قیمت مسکن) مراحل اصلی یک پروژه یادگیری ماشین را بررسی کنیم



- نگاه به تصویر بزرگ (big picture)

- دریافت داده‌ها

- بررسی و مصورسازی داده‌ها

- آماده‌سازی داده‌ها برای الگوریتم‌های یادگیری ماشین

- انتخاب مدل و آموزش آن

- تنظیم دقیق مدل

- ارائه راه‌حل پیشنهادی

- راه‌اندازی، نظارت و نگهداری سیستم

آماده‌سازی داده برای الگوریتم‌های یادگیری ماشین

- ابتدا مقدار هدف را از سایر ویژگی‌ها جدا می‌کنیم

```
housing = strat_train_set.drop("median_house_value", axis=1)
housing_labels = strat_train_set["median_house_value"].copy()
```

- سپس داده‌ها را پاک‌سازی می‌کنیم

- اکثر الگوریتم‌های یادگیری ماشین نمی‌توانند با ویژگی‌های مفقود (missing features) کار کنند
 - برای محله‌هایی که تعداد اتاق‌های خواب مشخص نشده است می‌توان:

```
housing.dropna(subset=["total_bedrooms"], inplace=True) # option 1
```

- این محله‌ها را حذف کرد

```
housing.drop("total_bedrooms", axis=1) # option 2
```

- این ویژگی را حذف کرد

```
median = housing["total_bedrooms"].median() # option 3
housing["total_bedrooms"].fillna(median, inplace=True)
```

- برای مقادیر نامشخص، مقداری را جایگزین کرد (imputation)

- صفر، میانگین، میانه، پیش‌بینی بر اساس ویژگی‌های دیگر

جانمایی (imputation)

```
from sklearn.impute import SimpleImputer
```

```
imputer = SimpleImputer(strategy="median")
```

```
housing_num = housing.select_dtypes(include=[np.number])
```

```
imputer.fit(housing_num)
```

```
>>> imputer.statistics_  
array([-118.51 , 34.26 , 29. , 2125. , 434. , 1167. , 408. , 3.5385])
```

```
>>> housing_num.median().values  
array([-118.51 , 34.26 , 29. , 2125. , 434. , 1167. , 408. , 3.5385])
```

```
X = imputer.transform(housing_num)
```

```
housing_tr = pd.DataFrame(X, columns=housing_num.columns,  
                           index=housing_num.index)
```

- میانه فقط روی ویژگی‌های عددی قابل اعمال است

- می‌توان از مدل‌های پیشرفته‌تر مانند KNNImputer استفاده کرد

مدیریت ویژگی‌های متنی

```
>>> housing_cat = housing[["ocean_proximity"]]
```

```
>>> housing_cat.head(8)
```

	ocean_proximity
13096	NEAR BAY
14973	<1H OCEAN
3785	INLAND
14689	INLAND
20507	NEAR OCEAN
1286	INLAND
18078	<1H OCEAN
4396	NEAR BAY

```
from sklearn.preprocessing import OrdinalEncoder
```

```
ordinal_encoder = OrdinalEncoder()
```

```
housing_cat_encoded = ordinal_encoder.fit_transform(housing_cat)
```

```
>>> ordinal_encoder.categories_
```

```
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],  
      dtype=object)]
```

- نزدیکی تا اقیانوس به صورت متنی ثبت شده است

- در واقع متن دلخواه نیستند بلکه یکی از چند دسته مشخص هستند

- بیشتر الگوریتم‌های یادگیری ماشین با داده‌های عددی کار می‌کنند

- به طور مثال می‌توانیم به هر دسته یک کد عددی اختصاص بدهیم

```
>>> housing_cat_encoded[:8]
```

```
array([[3.],  
       [0.],  
       [1.],  
       [1.],  
       [4.],  
       [1.],  
       [0.],  
       [3.]])
```

مدیریت ویژگی‌های متنی

```
>>> housing_cat = housing[["ocean_proximity"]]
```

```
>>> housing_cat.head(8)
```

	ocean_proximity
13096	NEAR BAY
14973	<1H OCEAN
3785	INLAND
14689	INLAND
20507	NEAR OCEAN
1286	INLAND
18078	<1H OCEAN
4396	NEAR BAY

- نزدیکی تا اقیانوس به صورت متنی ثبت شده است

- در واقع متن دلخواه نیستند بلکه یکی از چند دسته مشخص هستند

- بیشتر الگوریتم‌های یادگیری ماشین با داده‌های عددی کار می‌کنند

- به طور مثال می‌توانیم به هر دسته یک کد عددی اختصاص بدهیم

- فاصله کدها باید متناسب با فاصله معنایی دسته‌ها باشد

- در این مثال، <1H OCEAN و NEAR OCEAN به هم مشابه هستند اما کدهای ۰ و ۴ را

- دارند و INLAND کد ۱ دارد

```
from sklearn.preprocessing import OrdinalEncoder
```

```
ordinal_encoder = OrdinalEncoder()
```

```
housing_cat_encoded = ordinal_encoder.fit_transform(housing_cat)
```

```
>>> ordinal_encoder.categories_
```

```
[array(['<1H OCEAN', 'INLAND', 'ISLAND', 'NEAR BAY', 'NEAR OCEAN'],  
      dtype=object)]
```

مدیریت ویژگی‌های متنی

```
>>> housing_cat = housing[["ocean_proximity"]]
```

```
>>> housing_cat.head(8)
```

	ocean_proximity
13096	NEAR BAY
14973	<1H OCEAN
3785	INLAND
14689	INLAND
20507	NEAR OCEAN
1286	INLAND
18078	<1H OCEAN
4396	NEAR BAY

- یک راه رایج این است که کدهایی به دسته‌ها اختصاص دهیم که فاصله دوبه‌دوی آنها مشابه باشد

- یک ویژگی باینری برای هر دسته

- مثلاً یک ویژگی این باشد که اگر "<1H OCEAN" بود، ۱ باشد و در غیر اینصورت ۰ باشد

- کدگذاری one-hot نامیده می‌شود

```
from sklearn.preprocessing import OneHotEncoder
```

```
cat_encoder = OneHotEncoder()
```

```
housing_cat_1hot = cat_encoder.fit_transform(housing_cat)
```

```
>>> housing_cat_1hot
```

```
<16512x5 sparse matrix of type '<class 'numpy.float64'>'
with 16512 stored elements in Compressed Sparse Row format>
```

```
>>> housing_cat_1hot.toarray()
```

```
array([[0., 0., 0., 1., 0.],
       [1., 0., 0., 0., 0.],
       [0., 1., 0., 0., 0.],
       ...,
       [0., 0., 0., 0., 1.],
       [1., 0., 0., 0., 0.],
       [0., 0., 0., 0., 1.]])
```

مدیریت ویژگی‌های دسته‌ای

• اگر یک ویژگی تعداد زیادی دسته داشته باشد، کدگذاری one-hot منجر به تعداد زیادی ویژگی می‌شود

- باعث کاهش سرعت آموزش و تنزل عملکرد عمل می‌شود

- می‌توان آن را با ویژگی‌های عددی مفید مرتبط با دسته‌ها جایگزین کرد

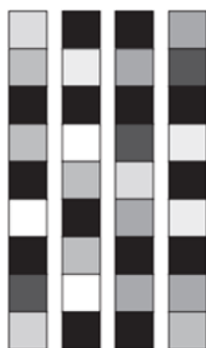
▪ مثلاً بجای کد یک کشور، ویژگی‌هایی مانند جمعیت کشور و تولید ناخالص داخلی جایگزین شود

- می‌توان از کدگذار قابل آموزش استفاده کرد



One-hot word vectors:

- Sparse
- High-dimensional
- Hardcoded



Word embeddings:

- Dense
- Lower-dimensional
- Learned from data

تبدیل ویژگی

- یکی از مهم ترین تبدیل‌هایی که داده‌ها به طور معمول اعمال می‌شود، تغییر مقیاس ویژگی‌ها است
- به جز چند استثناء، الگوریتم‌های یادگیری ماشین زمانی که ویژگی‌های عددی ورودی مقیاس‌های بسیار متفاوتی دارند، عملکرد خوبی ندارند
- برای داده‌های مسکن: تعداد کل اتاق‌ها از حدود ۶ تا ۳۹۳۲۰ اتاق متغیر است، در حالی که درآمد متوسط فقط از ۰ تا ۱۵ متغیر است
- دو راه متداول برای بدست آوردن مقیاس یکسان برای همه ویژگی‌ها وجود دارد:

```
from sklearn.preprocessing import MinMaxScaler
```

```
min_max_scaler = MinMaxScaler(feature_range=(-1, 1))
```

```
housing_num_min_max_scaled = min_max_scaler.fit_transform(housing_num)
```

```
from sklearn.preprocessing import StandardScaler
```

```
std_scaler = StandardScaler()
```

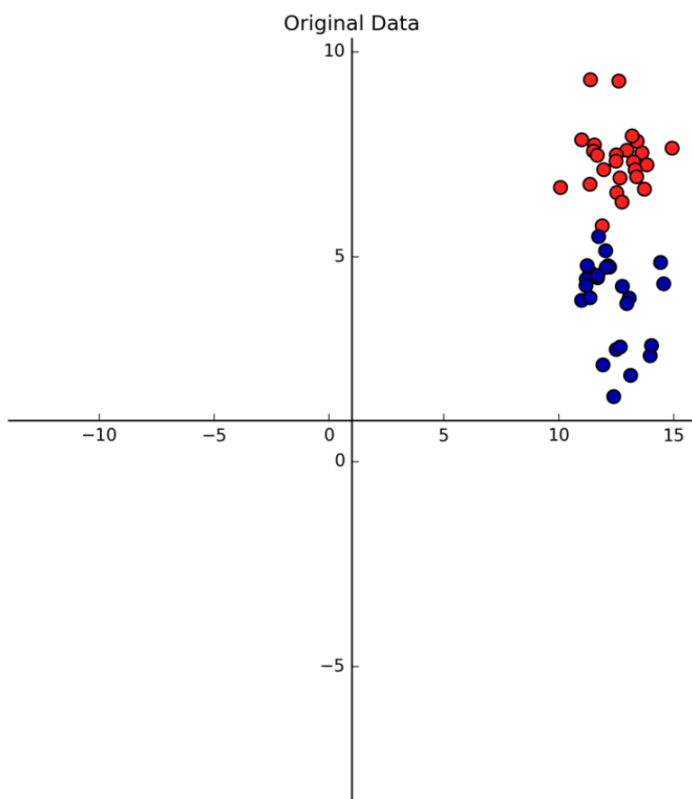
```
housing_num_std_scaled = std_scaler.fit_transform(housing_num)
```

- نرمال‌سازی minmax (حداقل ۰ حداکثر ۱)

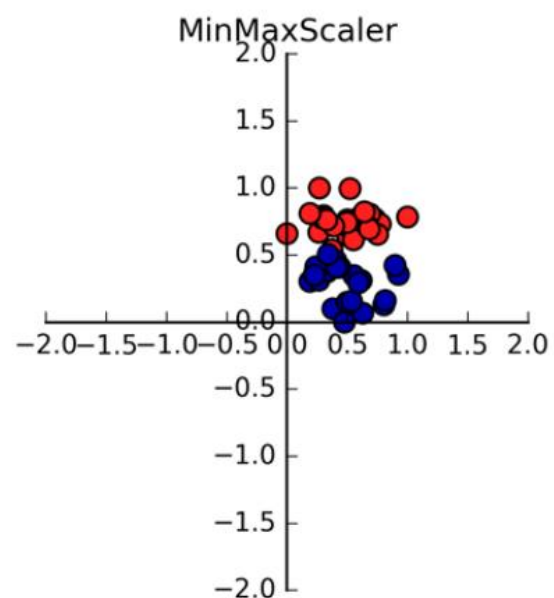
- استانداردسازی (میانگین ۰ واریانس ۱)

▪ حساسیت کمتر به داده‌های پرت

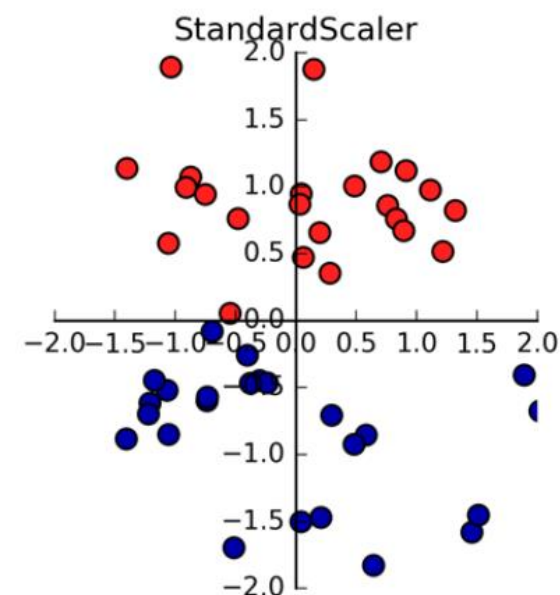
تبدیل ویژگی



$$x = \frac{x - X_{min}}{X_{max} - X_{min}}$$

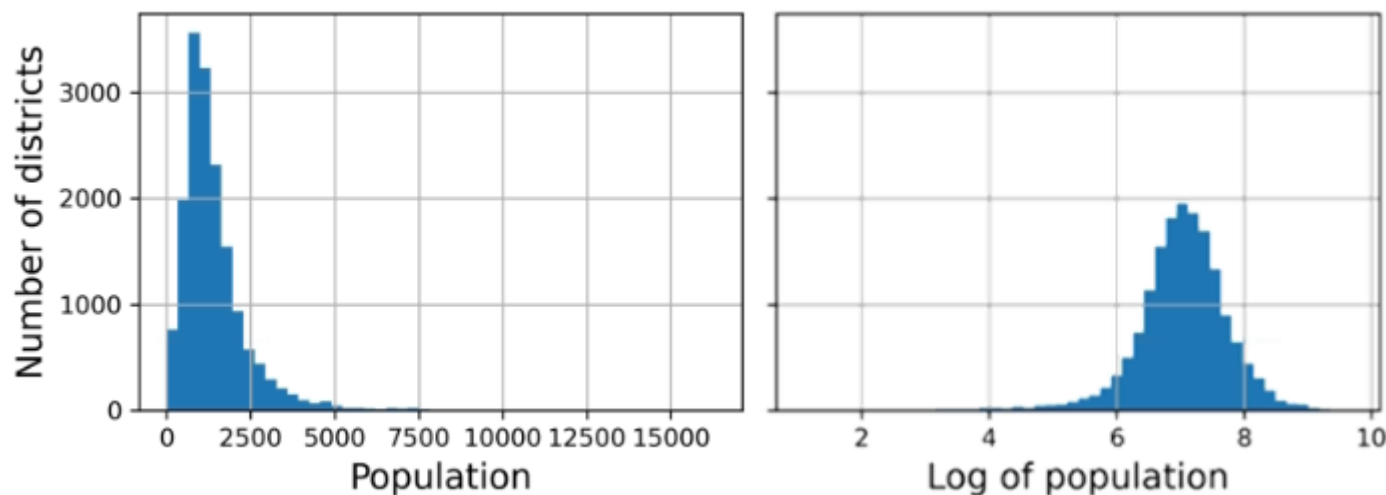


$$x = \frac{x - \mu}{\sigma}$$



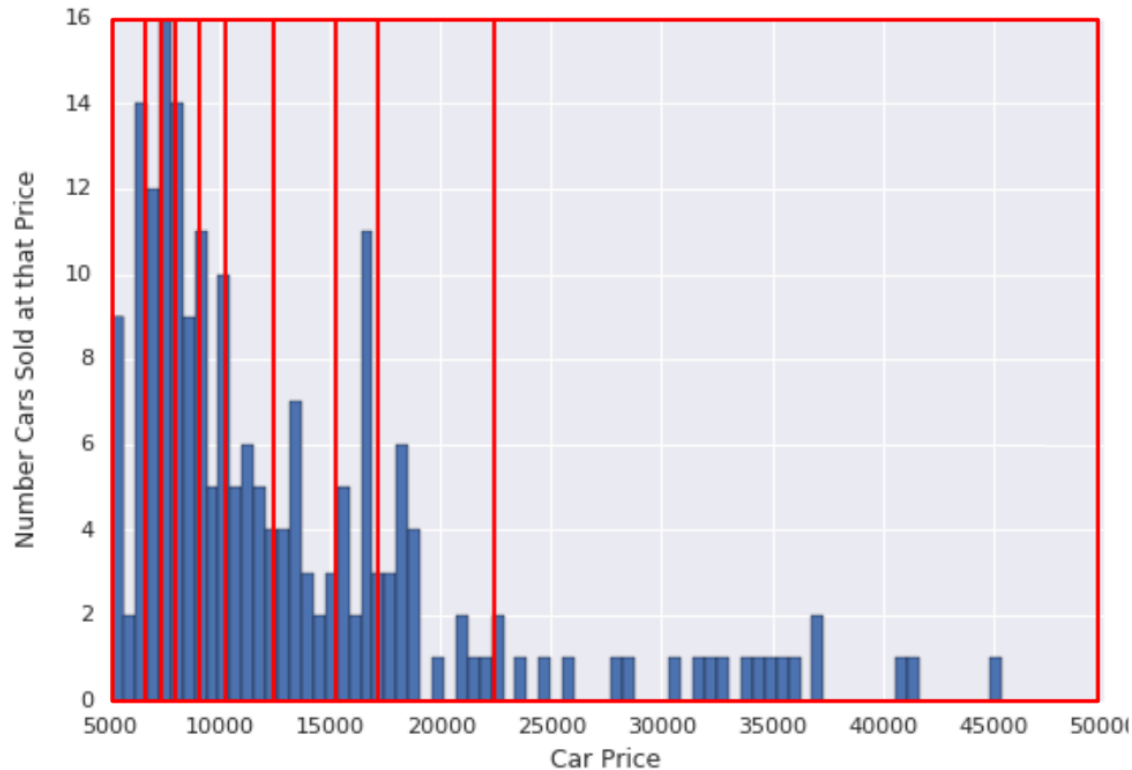
تبدیل ویژگی

- اگر توزیع یک ویژگی دُم‌بلند (heavy tailed) باشد (مقادیر دور از میانگین به طور نمایی نادر نباشند)
- تبدیل‌های قبل باعث می‌شوند بیشتر مقادیر در یک بازه کوچک قرار بگیرند
 - الگوریتم‌های یادگیری ماشین معمولاً چنین چیزی را دوست ندارند
- قبل از تغییر مقیاس، می‌توان با استفاده از یک تبدیل غیرخطی توزیع را متقارن و شبیه به نرمال کرد
 - در این مثال می‌توان از تابع لگاریتم استفاده کرد

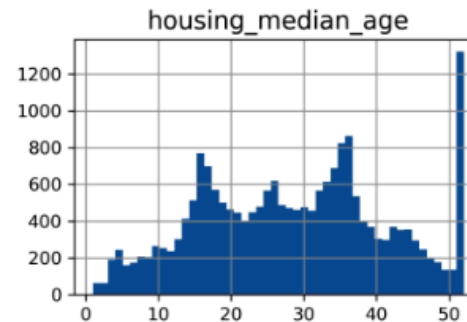


سطل‌بندی (bucketizing)

- یک رویکرد دیگر برای مدیریت ویژگی‌های دُم‌بلند است که توزیع را به چند سطل مشابه تقسیم می‌کند و به هر کدام یک مقدار اختصاص می‌دهد



- برای ویژگی‌های دارای توزیع چندوجهی (multimodal) (با چند قله واضح) هم مناسب است
- می‌توان برای آنها از کدگذاری one-hot استفاده کرد
- مثلاً ممکن است فقط زیاد بودن سال ساخت مهم نباشد بلکه سبک ساخت مهم باشد



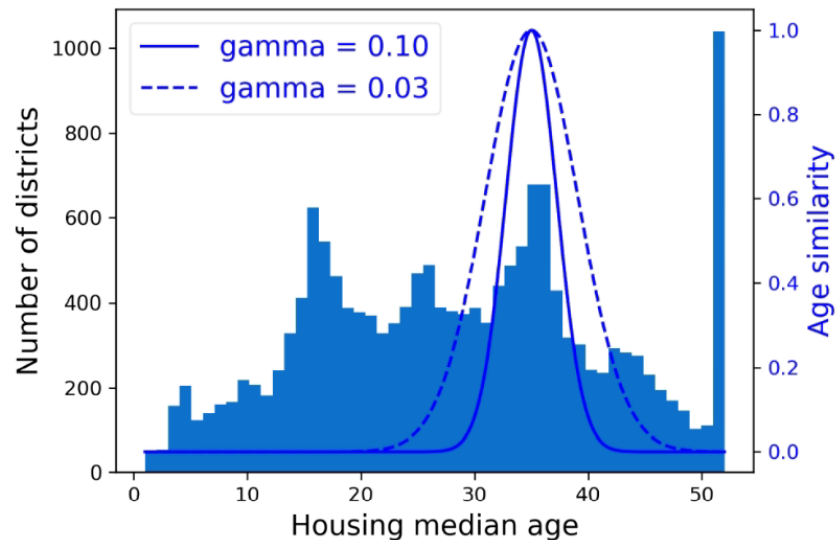
تبدیل ویژگی

- می‌توان متناسب با هر کدام از mode ها یک ویژگی تعریف کرد که مشابهت با آن را محاسبه کنند
 - یکی از توابع معروف برای این منظور تابع پایه شعاعی (RBF) است

▪ معروفترین آنها تابع RBF گاوسی است

```
from sklearn.metrics.pairwise import rbf_kernel
```

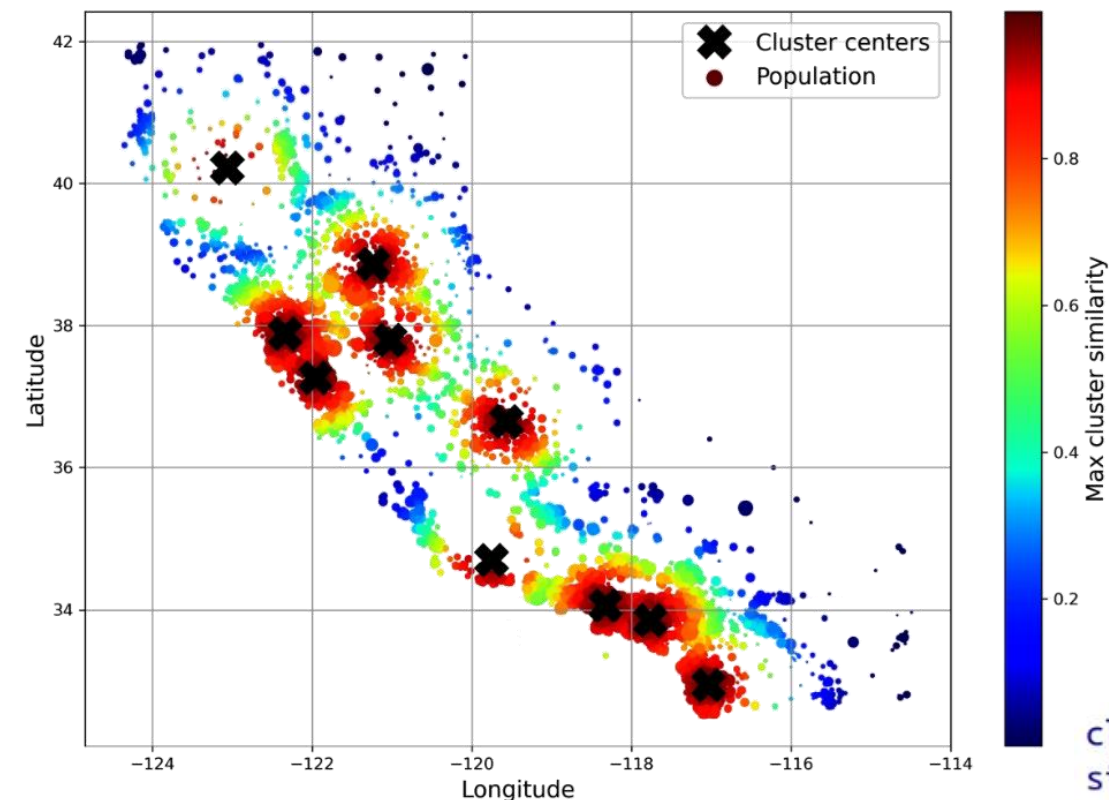
```
age_simil_35 = rbf_kernel(housing[["housing_median_age"]], [[35]], gamma=0.1)
```



$$\exp(-\gamma(x - 35)^2)$$

تبدیل ویژگی

- می‌توان با استفاده از خوشه‌بندی، mode‌های اصلی میان چند ویژگی را تشخیص داده و فاصله تا مراکز خوشه‌ها را به عنوان ویژگی انتخاب کرد



```
cluster_simil = ClusterSimilarity(n_clusters=10, gamma=1., random_state=42)
similarities = cluster_simil.fit_transform(housing[["latitude", "longitude"]],
                                          sample_weight=housing_labels)
```

```
>>> similarities[:3].round(2)
array([[0.  , 0.14, 0.  , 0.  , 0.  , 0.08, 0.  , 0.99, 0.  , 0.6 ],
       [0.63, 0.  , 0.99, 0.  , 0.  , 0.  , 0.04, 0.  , 0.11, 0.  ],
       [0.  , 0.29, 0.  , 0.  , 0.01, 0.44, 0.  , 0.7 , 0.  , 0.3 ]])
```

تبدیل هدف

```
from sklearn.linear_model import LinearRegression

target_scaler = StandardScaler()
scaled_labels = target_scaler.fit_transform(housing_labels.to_frame())

model = LinearRegression()
model.fit(housing[["median_income"]], scaled_labels)

# pretend this is new data
some_new_data = housing[["median_income"]].iloc[:5]

scaled_predictions = model.predict(some_new_data)
predictions = target_scaler.inverse_transform(scaled_predictions)

from sklearn.compose import TransformedTargetRegressor

model = TransformedTargetRegressor(LinearRegression(),
                                   transformer=StandardScaler())
model.fit(housing[["median_income"]], housing_labels)
predictions = model.predict(some_new_data)
```

- اگر توزیع هدف مناسب نباشد، می‌توان مقادیر هدف را هم تبدیل کرد
- بعد از پیش‌بینی، لازم است معکوس تبدیل روی آن اعمال شود
- می‌توان با توابع sklearn این کار را ساده‌تر و در یک مرحله انجام داد

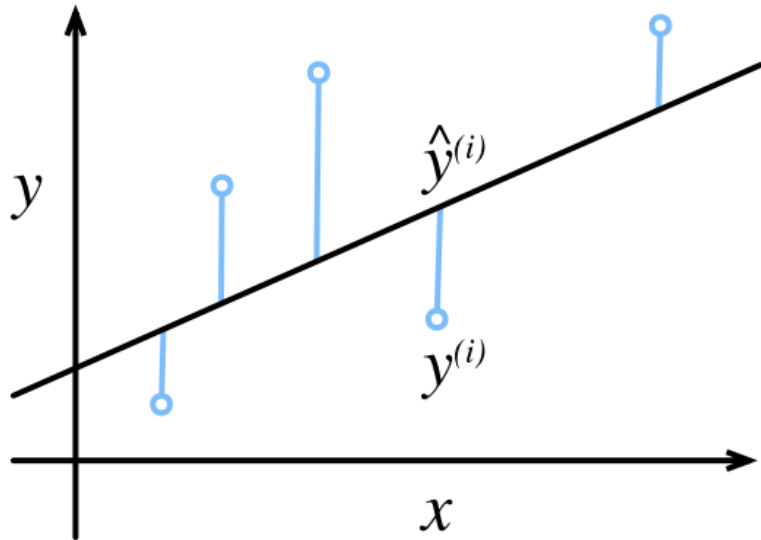
انتخاب و آموزش یک مدل

- رگرسیون خطی یک مدل ساده برای تخمین یک عدد پیوسته است

$$\hat{y} = w_1x_1 + \dots + w_dx_d + b$$

- مدل‌های خطی بر فرض یکنواختی دلالت دارند

- هر افزایشی در ویژگی همواره منجر به افزایش (کاهش) خروجی متناسب می‌شود اگر وزن مربوطه + (-) باشد



آموزش رگرسیون خطی

```
from sklearn.linear_model import LinearRegression
```

```
lin_reg = make_pipeline(preprocessing, LinearRegression())  
lin_reg.fit(housing, housing_labels)
```

```
>>> housing_predictions = lin_reg.predict(housing)  
>>> housing_predictions[:5].round(-2) # -2 = rounded to tl  
array([243700., 372400., 128800., 94400., 328300.])  
>>> housing_labels.iloc[:5].values  
array([458300., 483800., 101700., 96100., 361800.])
```

```
>>> from sklearn.metrics import mean_squared_error  
>>> lin_rmse = mean_squared_error(housing_labels, housing_predictions,  
...                               squared=False)  
...  
...  
>>> lin_rmse  
68687.89176589991
```

• نتیجه چندان خوب نیست!

- در شرایطی که اکثر محله‌ها دارای قیمت در بازه 120,000\$ و 265,000\$ هستند، خطای پیش‌بینی 68,687\$ زیاد است
- مدل underfit شده است
- می‌توان از مدل قوی‌تر و/یا ویژگی‌های بهتر استفاده کرد